Cloud Computing Complete Interview Reference — Full Edition

> **All topics merged:** Cloud Fundamentals · Architecture · AWS · Azure · GCP · Security · DevOps · Networking · Containers · Observability · Advanced Every section includes subtopics, keywords, and interview notes.

TABLE OF CONTENTS

Foundations

- 1. Cloud Fundamentals
- 2. Cloud Service & Deployment Models
- 3. Cloud Economics & Pricing
- 4. Cloud Security Fundamentals
- 5. Cloud Networking Fundamentals

Architecture

- 6. Cloud Architecture Principles
- 7. Scalability & Resilience Patterns
- 8. Distributed Systems in the Cloud
- 9. Serverless Architecture
- 10. Container-Based Architecture
- 11. Event-Driven & Microservices Architecture

AWS

- 12. AWS Core Concepts & Infrastructure
- 13. AWS Compute
- 14. AWS Storage
- 15. AWS Networking
- 16. AWS Databases
- 17. AWS Security

- 18. AWS Monitoring & DevOps
- 19. AWS Advanced Patterns

Azure

- 20. Azure Core Concepts & Infrastructure
- 21. Azure Compute
- 22. Azure Storage
- 23. Azure Networking
- 24. Azure Databases
- 25. Azure Security
- 26. Azure Monitoring & DevOps
- 27. Azure Advanced Patterns

GCP

- 28. GCP Core Concepts & Infrastructure
- 29. GCP Compute
- 30. GCP Storage & Databases
- 31. GCP Networking & Security
- 32. GCP Monitoring & DevOps
- 33. GCP Advanced Patterns

Cross-Cloud & Operations

- 34. AWS vs Azure vs GCP Comparison
- 35. Multi-Cloud & Hybrid Cloud
- 36. Cloud DevOps & CI/CD
- 37. Cloud Observability & Monitoring
- 38. Cloud Cost Optimization
- 39. Disaster Recovery & High Availability
- 40. Cloud Compliance & Governance

- 41. Cloud Anti-Patterns
- 42. Interview Master Keywords

---

1. Cloud Fundamentals

Core Concepts

- **Cloud Computing** → On-demand delivery of IT resources over the internet; pay-as-you-go; no upfront hardware investment.
- **NIST Definition** → On-demand self-service, broad network access, resource pooling, rapid elasticity, measured service (5 essential characteristics).
- **Elasticity** → Automatically scale resources up/down with demand; key differentiator from traditional hosting.
- **Scalability** → Ability to handle growth; vertical (scale-up) or horizontal (scale-out).
- **High Availability (HA)** → System remains operational despite component failures; measured in nines (99.9% = 8.7h/year downtime).
- **Fault Tolerance** → System continues operating even when components fail; zero or near-zero downtime.
- **Regions** → Geographically separated data center clusters; data sovereignty, latency, compliance drive choice.
- **Availability Zones (AZs)** → Isolated data centers within a region; connected by low-latency links; failure domain boundaries.
- **Edge Locations** → CDN PoPs closer to users; CloudFront, Azure CDN, Cloud CDN; reduce latency for static/cached content.
- **Shared Responsibility Model** → Cloud provider secures infrastructure; customer secures data, identity, application, config.
- **Managed vs Unmanaged Services** → Managed = provider handles patching/scaling/HA; Unmanaged = you control everything (EC2 vs Lambda).

Migration Strategies (6 Rs)

- **Rehost (Lift-and-Shift)** → Move as-is to cloud VMs; fastest; least optimized; good for quick exit from data center.
- **Replatform** → Minor cloud optimizations (move DB to RDS); minimal code change; moderate benefit.
- **Repurchase** → Switch to SaaS alternative (Salesforce instead of on-prem CRM).
- **Refactor / Re-architect** → Redesign for cloud-native (monolith → microservices); most benefit, most effort.

- **Retire** → Decommission unused applications; reduces licensing cost.
- **Retain** → Keep on-premise for now; compliance, latency, or cost reasons.
- **Keywords**: Lift-and-shift = fast migration, no optimization; re-architect = cloud-native, full benefit; choose based on business timeline and risk tolerance.

Cloud Adoption Frameworks

- **AWS CAF** → Six perspectives: Business, People, Governance, Platform, Security, Operations.
- **Azure CAF** → Strategy, Plan, Ready, Adopt, Govern, Manage phases.
- **Google Cloud Adoption Framework** → Learn, Lead, Scale, Secure themes.
- **Vendor Lock-in** → Dependency on provider-specific services; mitigate with open standards, abstraction layers, multi-cloud.

Interview Deep-Dive Keywords

- "Elasticity is automatic — scalability is a design property; both are needed"
- "AZs are the HA building block — deploy across ≥2 AZs for production workloads"
- "Shared responsibility: provider secures the cloud; you secure what's in the cloud"
- "Lift-and-shift gets you to cloud fast but leaves cost and performance benefits on the table"

---

2. Cloud Service & Deployment Models

Service Models

- **IaaS (Infrastructure as a Service)** → Virtualized compute, storage, network; you manage OS + up; EC2, Azure VMs, Compute Engine.
- **PaaS (Platform as a Service)** → Runtime + middleware + OS managed; you manage app + data; Elastic Beanstalk, App Service, App Engine.
- **SaaS (Software as a Service)** → Full application managed by provider; you manage users/data only; Salesforce, Gmail, Office 365.
- **FaaS (Function as a Service)** → Event-triggered function execution; you manage only code; Lambda, Azure Functions, Cloud Functions.
- **CaaS (Container as a Service)** → Managed container orchestration; EKS, AKS, GKE.
- **DBaaS (Database as a Service)** → Managed database; RDS, Cosmos DB, Cloud SQL.

Responsibility by Layer

Layer	IaaS	PaaS	SaaS
Application	Customer	Customer	Provider
Runtime	Customer	Provider	Provider
OS	Customer	Provider	Provider
Virtualization	Provider	Provider	Provider
Physical	Provider	Provider	Provider

Deployment Models

- **Public Cloud** → Shared infrastructure; multi-tenant; most cost-effective; AWS, Azure, GCP.
- **Private Cloud** → Dedicated infrastructure; single tenant; on-prem or hosted; more control, higher cost.
- **Hybrid Cloud** → Mix of public + private; sensitive data on-prem, workloads on public; complexity vs control.
- **Multi-Cloud** → Multiple public cloud providers; avoid lock-in, best-of-breed; operational complexity.
- **Community Cloud** → Shared by organizations with common concerns (government, healthcare); rare.

Interview Deep-Dive Keywords

- "IaaS = most control, most responsibility; SaaS = least control, least responsibility"
- "Hybrid cloud is not multi-cloud — hybrid = on-prem + cloud; multi-cloud = multiple public clouds"
- "PaaS accelerates development but can introduce lock-in (e.g. App Engine-specific APIs)"
- "FaaS is the ultimate PaaS — no server management at all, but cold starts and timeout limits apply"

---

3. Cloud Economics & Pricing

Pricing Models

- **Pay-as-you-go (PAYG)** → Pay only for what you use; no upfront; highest per-unit cost; best for unpredictable workloads.

- **Reserved Instances (RI)** → 1 or 3-year commitment; 30–72% discount vs on-demand; best for stable baseline.
- **Savings Plans** → Flexible commitment to $/hr spend; covers EC2, Lambda, Fargate; easier than RIs.
- **Spot / Preemptible Instances** → Unused capacity at 60–90% discount; can be interrupted with 2-min notice; stateless/batch workloads only.
- **Committed Use Discounts (GCP)** → 1 or 3-year resource commitment; automatic discount for eligible usage.
- **Free Tier** → AWS/Azure/GCP all offer free tiers; useful for dev/test; watch expiry dates.

Cost Drivers

- **Compute** → Instance type, hours running, data transfer out, licensing.
- **Storage** → GB stored, I/O operations, data retrieval (Glacier), replication.
- **Data Transfer** → Ingress usually free; egress (out to internet) is charged; cross-region transfer charged.
- **Managed Services** → Premium over self-managed; pays for reduced operational overhead.
- **Idle Resources** → Stopped EC2 still charges for EBS; idle RDS still charges; biggest waste source.

Cost Optimization Strategies

- **Right-Sizing** → Match instance size to actual utilization; use CloudWatch/Azure Monitor metrics.
- **Auto Scaling** → Scale in when load drops; eliminate over-provisioning.
- **Reserved + Spot Mix** → Reserve baseline, use spot for burst; Spot Fleet / Spot Instances with on-demand fallback.
- **Storage Tiering** → S3 Intelligent-Tiering, Glacier for cold data; lifecycle policies auto-transition.
- **Delete Unused Resources** → Unattached EBS volumes, idle load balancers, old snapshots, unused EIPs.
- **Tagging** → Tag every resource with team/project/env; mandatory for cost allocation reports.
- **FinOps** → Practice of cloud financial management; shared accountability across engineering + finance.
- **Keywords**: Egress fees are the hidden cloud tax; Reserved Instances for steady state, Spot for burst; tagging is the prerequisite to cost visibility.

Interview Deep-Dive Keywords

- "Spot instances are great for stateless, fault-tolerant batch jobs — not for databases"

- "Reserved Instances save money but require capacity planning — use Savings Plans for flexibility"
- "The biggest cloud waste is running resources 24/7 that are only needed 8 hours/day"
- "FinOps = shift left on cost — engineers own cost as much as reliability"

---

4. Cloud Security Fundamentals

Identity & Access Management

- **IAM** → Who can do what on which resource; users, roles, policies, groups.
- **Principle of Least Privilege** → Grant minimum permissions needed; revoke unused access.
- **Role-Based Access Control (RBAC)** → Permissions assigned to roles; users assume roles; scalable.
- **Attribute-Based Access Control (ABAC)** → Policies based on resource tags/attributes; dynamic; fine-grained.
- **Service Account / Role** → Identity for applications/services; no human; short-lived credentials.
- **MFA (Multi-Factor Authentication)** → Require second factor for sensitive operations; mandatory for root/admin.
- **Federation** → Use existing IdP (Active Directory, Okta) with cloud IAM; SSO; SAML/OIDC.
- **Cross-Account Roles** → Assume a role in another AWS account; avoids long-lived cross-account credentials.

Shared Responsibility Model (Detail)

- **Provider Responsibility** → Physical security, hypervisor, network fabric, managed service OS patching.
- **Customer Responsibility (IaaS)** → OS patching, firewall config, application security, data encryption, IAM.
- **Customer Responsibility (PaaS/SaaS)** → Data classification, access control, configuration security.
- **Misconfiguration** → #1 cloud breach cause; public S3 buckets, open security groups, overpermissive IAM.

Key Security Controls

- **Encryption at Rest** → KMS-managed keys; enforce via bucket/disk policies; default encryption on all storage.
- **Encryption in Transit** → TLS 1.2+ for all APIs; enforce HTTPS-only; no plaintext endpoints.

- **Secrets Management** → AWS Secrets Manager, Azure Key Vault, GCP Secret Manager; never hard-code credentials.
- **Network Security** → Security groups (stateful), NACLs (stateless), WAF, DDoS protection.
- **Audit Logging** → CloudTrail, Azure Activity Log, GCP Audit Logs; immutable; enable in all regions.
- **Vulnerability Management** → AWS Inspector, Defender for Cloud, Security Command Center; automated scanning.
- **Zero Trust** → Never trust, always verify; assume breach; network location ≠ trust; identity-first.

Interview Deep-Dive Keywords

- "Misconfiguration is the #1 cloud security risk — not hacking; S3 bucket policies, SGs, IAM are the attack surface"
- "Root account = nuclear option; use only to create first admin user, then lock it away with MFA"
- "Service roles > long-lived API keys; rotate anything that must be static; use secrets manager"
- "Zero trust in cloud: identity + device + context = access decision, not network perimeter"

---

5. Cloud Networking Fundamentals

Virtual Networks

- **VPC (AWS) / VNet (Azure) / VPC (GCP)** → Isolated virtual network; your private space in the cloud.
- **Subnet** → Logical subdivision of VPC; public (internet-accessible) or private (internal only).
- **CIDR Block** → IP address range for VPC/subnet; plan carefully (e.g. 10.0.0.0/16); cannot overlap with peered VPCs.
- **Route Table** → Rules directing traffic between subnets, internet, VPN.
- **Internet Gateway (IGW)** → Enables internet access from public subnets; AWS-specific term.
- **NAT Gateway** → Allows private subnet instances to initiate outbound internet traffic; blocks inbound; stateful.
- **Security Group** → Stateful virtual firewall at instance/resource level; allow rules only; AWS/Azure.
- **Network ACL (NACL)** → Stateless firewall at subnet level; allow + deny rules; processed in order.

- **VPC Peering** → Private connectivity between two VPCs; no transitive peering (A↔B, B↔C ≠ A↔C).
- **Transit Gateway (AWS) / VNet Peering + Hub (Azure)** → Central hub for connecting many VPCs/VNets; transitive routing.
- **Private Endpoints** → Access cloud services (S3, Key Vault) over private IP, not internet; VPC Endpoint (AWS), Private Link (Azure).

Connectivity

- **VPN Gateway** → Encrypted tunnel over internet; site-to-site (on-prem) or client VPN; lower cost than Direct Connect.
- **Direct Connect (AWS) / ExpressRoute (Azure) / Interconnect (GCP)** → Dedicated private circuit from on-prem to cloud; consistent bandwidth; lower latency than VPN; higher cost.
- **CDN** → CloudFront, Azure CDN, Cloud CDN; cache content at edge locations; reduce origin load and latency.
- **DNS** → Route 53 (AWS), Azure DNS, Cloud DNS; managed DNS; health-check-based routing.
- **Load Balancer** → Distribute traffic across instances; Application LB (L7 HTTP), Network LB (L4 TCP/UDP).

Interview Deep-Dive Keywords

- "Public subnet = has route to internet gateway; private subnet = no direct internet route"
- "Security groups are stateful — return traffic automatically allowed; NACLs are stateless — need inbound + outbound rules"
- "VPC peering is not transitive — use Transit Gateway for hub-and-spoke topology"
- "NAT Gateway is for outbound-only from private subnets; never expose DB or internal services to a public subnet"
- "Direct Connect vs VPN: Direct Connect = dedicated bandwidth + consistent latency + high cost; VPN = cheap + setup fast + variable latency"

---

6. Cloud Architecture Principles

Well-Architected Pillars

- **Operational Excellence** → Run and monitor systems; automate operations; learn from failures; IaC everything.
- **Security** → Protect data, systems, assets; apply security at all layers; automate security response.

- **Reliability** → Recover from failures; automatically heal; test recovery procedures; chaos engineering.
- **Performance Efficiency** → Use resources efficiently; select right resource types; monitor performance; innovate with serverless/managed.
- **Cost Optimization** → Avoid unnecessary costs; right-size; use managed services; analyze spend.
- **Sustainability** → Minimize environmental impact; right-size; use efficient regions; maximize utilization.

Design Principles

- **Design for Failure** → Assume any component will fail; build redundancy; never single point of failure.
- **Loose Coupling** → Minimize dependencies between components; use queues/events; failure isolation.
- **Stateless Services** → Store state externally (Redis, DB); any instance can handle any request; enables horizontal scaling.
- **Disposability** → Instances are cattle not pets; terminate/replace freely; immutable infrastructure.
- **Automation First** → Nothing done manually in production; IaC for infrastructure; pipelines for deployments.
- **Defense in Depth** → Multiple security layers; assume one layer will fail; WAF + SG + IAM + encryption.
- **Immutable Infrastructure** → Never modify running instances; replace entire image; eliminates config drift.
- **Infrastructure as Code (IaC)** → Terraform, CDK, Bicep; version-controlled; reproducible; auditable.

Patterns

- **Bulkhead** → Isolate components into pools; failure in one pool doesn't cascade; thread pools, connection pools per service.
- **Circuit Breaker** → Stop calling failing service; return fallback; attempt reset after timeout; prevents cascade failure.
- **Retry with Exponential Backoff** → Retry transient failures; add jitter to avoid thundering herd.
- **Throttling** → Limit request rate per client; protects downstream; API Gateway, Redis rate limiter.
- **Queue-Based Load Leveling** → Buffer requests in queue (SQS, Service Bus); decouple producer speed from consumer speed.

- **Competing Consumers** → Multiple workers on same queue; parallel processing; scale workers independently.
- **Sidecar** → Deploy helper container alongside main service; logging, service mesh, monitoring agent.

Interview Deep-Dive Keywords

- "Design for failure = assume AZ will go down; design so system survives it without manual intervention"
- "Stateless is the prerequisite for horizontal scaling — if your app needs sticky sessions, you're not stateless"
- "Circuit breaker prevents cascade failure — without it, one slow service can take down the whole system"
- "IaC is not optional at scale — manual changes cause drift, are unauditable, and can't be rolled back"

---

7. Scalability & Resilience Patterns

Scalability

- **Vertical Scaling (Scale-Up)** → Larger instance; simple but has ceiling; no code change; downtime for in-place resize.
- **Horizontal Scaling (Scale-Out)** → More instances; no ceiling; requires stateless app design; preferred for cloud.
- **Auto Scaling** → Automatically add/remove instances based on metrics (CPU, request rate, queue depth).
    - **Target Tracking** → Maintain a metric at a target value (e.g. CPU = 60%); simplest.
    - **Step Scaling** → Add N instances per threshold breach step.
    - **Scheduled Scaling** → Pre-scale for known traffic events (sales, launches).
    - **Predictive Scaling** → ML-based forecasting; AWS feature; scale before traffic arrives.
- **Scale-In Protection** → Prevent Auto Scaling from terminating instances with active work.
- **Warm Pool (AWS)** → Pre-initialized stopped instances; faster scale-out than cold launch.

Load Balancing

- **Application Load Balancer (ALB / L7)** → HTTP/HTTPS; path/header routing; WebSockets; sticky sessions; WAF integration.
- **Network Load Balancer (NLB / L4)** → TCP/UDP/TLS; ultra-low latency; static IPs; millions of requests/sec.

- **Classic Load Balancer** → Legacy; avoid for new workloads.
- **Global Load Balancer** → Route traffic across regions; AWS Global Accelerator, Azure Front Door, Cloud Load Balancing.
- **Health Checks** → LB routes only to healthy targets; fail fast on unhealthy instances.
- **Sticky Sessions** → Route same user to same instance; avoid if possible (breaks horizontal scale).

Resilience

- **Multi-AZ Deployment** → Spread across ≥2 AZs; survive single AZ failure; standard for all production.
- **Multi-Region Deployment** → Survive regional failure; active-active or active-passive; much higher complexity.
- **RTO / RPO** → Recovery Time Objective (how long down); Recovery Point Objective (how much data loss); drive HA tier choice.
- **Chaos Engineering** → Deliberately inject failures; validate assumptions; Netflix Chaos Monkey, AWS Fault Injection Simulator.
- **Graceful Degradation** → Serve partial functionality when dependencies fail; better than complete outage.
- **Bulkhead Isolation** → Separate resource pools per consumer type; noisy neighbor can't consume all resources.

Interview Deep-Dive Keywords

- "Auto Scaling based on CPU alone is dangerous — use queue depth or request count for better signal"
- "ALB for HTTP workloads; NLB for raw TCP or when you need static IPs (firewalls, gaming)"
- "Multi-AZ ≠ high availability guarantee — also need health checks, connection draining, graceful shutdown"
- "RTO and RPO are business decisions — they drive your architecture tier, not the other way around"

---

8. Distributed Systems in the Cloud

Fundamentals

- **CAP Theorem** → Consistency, Availability, Partition Tolerance — pick 2; partition is unavoidable → choose CP or AP.

- **PACELC** → During partition: CA tradeoff; Else: Latency vs Consistency tradeoff; better model for cloud systems.
- **Eventual Consistency** → Writes propagate asynchronously; replicas converge; available during partition; design for it.
- **Idempotency** → Operation produces same result if executed multiple times; mandatory for at-least-once message delivery.
- **Distributed Tracing** → Track request across services via trace ID; OpenTelemetry, Jaeger, X-Ray, Zipkin.
- **Service Discovery** → Services find each other dynamically; DNS-based (ECS service discovery), client-side (Eureka), server-side (ALB).

Consistency Patterns

- **Saga Pattern** → Distributed transaction via local transactions + compensating transactions; choreography or orchestration.
- **Outbox Pattern** → Write event to DB outbox atomically; CDC/poller publishes to message broker; solves dual-write.
- **Two-Phase Commit (2PC)** → Distributed atomic commit; blocking; coordinator SPOF; avoid in modern microservices.
- **Event Sourcing** → Store state as immutable event log; replay to reconstruct; full audit trail.
- **CQRS** → Separate read model from write model; sync via events; optimize each side independently.

Communication Patterns

- **Synchronous (REST/gRPC)** → Request-response; simple; tight temporal coupling; caller blocks.
- **Asynchronous (Queue/Event)** → Producer decoupled from consumer; resilient; adds complexity; message ordering.
- **Message Queue** → Point-to-point; one consumer per message; SQS, Azure Service Bus Queue.
- **Pub/Sub** → One publisher, many subscribers; SNS, EventBridge, Azure Event Grid, Pub/Sub.
- **Backpressure** → Consumer signals producer to slow down; prevents overwhelming downstream.
- **Dead Letter Queue (DLQ)** → Capture messages that fail processing repeatedly; critical for debugging.

Interview Deep-Dive Keywords

- "Idempotency is non-negotiable in distributed systems — network retries are guaranteed to happen"
- "Saga replaces 2PC for microservices — eventual consistency with compensating transactions"
- "Outbox pattern is the safe way to combine DB write + message publish atomically"
- "CQRS shines when read and write patterns differ wildly — don't use it for simple CRUD"

---

9. Serverless Architecture

Core Concepts

- **Serverless** → No server management; auto-scale to zero; pay per invocation; provider manages all infrastructure.
- **FaaS** → Lambda, Azure Functions, Cloud Functions; event-triggered; stateless; ephemeral execution environment.
- **Cold Start** → Initial invocation spins up execution environment; adds latency (100ms–1s+); mitigate with Provisioned Concurrency.
- **Warm Start** → Reuse existing execution environment; sub-millisecond overhead.
- **Execution Timeout** → Lambda max 15 min; Azure Functions 10 min (Consumption); design for short-lived execution.
- **Concurrency** → Each invocation runs in its own environment; scales to 1000s simultaneously; watch DB connection limits.
- **Provisioned Concurrency** → Pre-warm N instances; eliminates cold starts; costs money even when idle.
- **Event Sources** → API Gateway, S3 events, SQS, SNS, DynamoDB Streams, EventBridge, Kinesis.

Patterns

- **Request-Response (Sync)** → API Gateway → Lambda; HTTP API patterns; cold start visible to user.
- **Event-Driven (Async)** → S3 upload → Lambda; SQS → Lambda; decoupled; retries built-in.
- **Fan-Out** → SNS → multiple Lambda; parallel processing of same event.
- **Orchestration** → Step Functions (AWS), Durable Functions (Azure); stateful workflows; retry/catch logic.

- **Strangler Fig** → Migrate monolith to serverless incrementally; route new features to Lambda, old to monolith.

Limitations

- **State** → Functions are stateless; use DynamoDB, S3, Redis for state.
- **Execution Duration** → Max timeout limits; not suitable for long-running jobs.
- **Cold Start** → Latency-sensitive APIs should use Provisioned Concurrency or keep-warm pings.
- **Concurrency Limits** → Regional account-level limit; request increase proactively.
- **Vendor Lock-in** → Lambda-specific triggers, Step Functions, etc; abstract via adapter layer if portability needed.
- **Keywords**: Serverless = operational simplicity, not zero cost; cold starts are the main trade-off for latency-sensitive workloads; concurrency × DB connections = danger.

Interview Deep-Dive Keywords

- "Cold starts hurt user-facing APIs — use Provisioned Concurrency or a lightweight runtime (Python/Node over Java)"
- "Lambda + RDS = connection pool exhaustion; use RDS Proxy to multiplex connections"
- "Step Functions for anything with multiple steps, retries, or human approval — don't build that in Lambda code"
- "Serverless is not always cheaper — high constant throughput workloads cost more than reserved EC2"

---

10. Container-Based Architecture

Core Concepts

- **Container** → Lightweight, portable isolated process; shares host OS kernel; faster than VMs.
- **Docker** → Build, ship, run containers; Dockerfile defines image; image = immutable artifact.
- **Container Image** → Layered filesystem; base OS + app layers; stored in registry (ECR, ACR, GCR, Docker Hub).
- **Container Registry** → Store and version container images; ECR (AWS), ACR (Azure), Artifact Registry (GCP).
- **Orchestration** → Manage container lifecycle, scheduling, networking at scale; Kubernetes dominant.

Kubernetes

- **Pod** → Smallest deployable unit; 1+ containers sharing network namespace + storage.
- **Deployment** → Manages desired number of Pod replicas; rolling updates; rollback.
- **Service** → Stable network endpoint for Pods; ClusterIP (internal), NodePort, LoadBalancer, ExternalName.
- **Ingress** → L7 routing into cluster; path/host-based rules; TLS termination; ALB Ingress Controller, Nginx.
- **ConfigMap / Secret** → Externalise configuration + credentials from image; Secrets should use KMS-backed store.
- **Persistent Volume (PV) / PVC** → Storage abstraction; EBS, EFS, Azure Disk, GCS backed.
- **Namespace** → Virtual cluster within cluster; tenant/environment isolation.
- **HPA (Horizontal Pod Autoscaler)** → Scale pod count based on CPU/memory/custom metrics.
- **VPA (Vertical Pod Autoscaler)** → Right-size pod CPU/memory requests and limits.
- **KEDA** → Event-driven autoscaling; scale on queue depth, Kafka lag, etc.
- **Node Pools** → Groups of nodes with same config; spot pools for batch, dedicated for prod.

Managed Kubernetes

- **EKS (AWS)** → Managed control plane; worker nodes on EC2/Fargate; Fargate profiles = serverless pods.
- **AKS (Azure)** → Managed; free control plane; VMSS-backed node pools; Azure AD integration.
- **GKE (GCP)** → Most mature; Autopilot mode = fully managed nodes; Workload Identity for GCP auth.
- **Fargate (AWS)** → Run containers without managing EC2; per-pod billing; good for variable/bursty workloads.

Service Mesh

- **Service Mesh** → Infrastructure layer for service-to-service communication; Istio, Linkerd, AWS App Mesh.
- **Features** → mTLS between services, traffic management (canary, A/B), observability (traces, metrics), retries/circuit breaking.
- **Sidecar Proxy** → Envoy proxy injected alongside each service container; intercepts all network traffic.
- **Keywords**: Container ≠ VM; containers share OS kernel; Kubernetes manages container lifecycle; service mesh manages service communication.

Interview Deep-Dive Keywords

- "Pods are ephemeral — never store state in pod filesystem; use PVC or external storage"
- "HPA + VPA together: HPA scales out, VPA right-sizes requests — don't run both in auto mode simultaneously"
- "Fargate eliminates node management but costs more than EC2 nodes at steady state; good for variable workloads"
- "Service mesh adds mTLS and observability but adds latency and operational complexity — worth it at scale"

---

11. Event-Driven & Microservices Architecture

Microservices

- **Microservices** → Small, independently deployable services; own data store; communicate via API/events.
- **Service Boundaries** → Defined by bounded context (DDD); one business capability per service.
- **Database per Service** → Each service owns its data; no shared DB; prevents coupling.
- **API Gateway** → Single entry point for clients; routing, auth, rate limiting, SSL termination; API GW, Kong, Apigee.
- **BFF (Backend for Frontend)** → Dedicated API gateway per client type (mobile, web); tailored response shapes.
- **Service Registry** → Track available service instances; Consul, Eureka, cloud-native DNS.
- **Distributed Tracing** → Correlate requests across services; trace ID propagated in headers.

Event-Driven Architecture

- **Event** → Immutable fact that something happened; `OrderPlaced`, `PaymentProcessed`.
- **Event Broker** → Route events between producers and consumers; Kafka, EventBridge, SNS/SQS, Azure Event Grid/Hubs.
- **Topic / Queue** → Topic = pub/sub (many consumers); Queue = point-to-point (one consumer).
- **Event Schema Registry** → Enforce event schema contracts; Glue Schema Registry, Confluent Schema Registry; Avro/Protobuf.
- **Choreography** → Services react to events; no central coordinator; decoupled; harder to trace.

- **Orchestration** → Central controller (Step Functions, Temporal) drives workflow; easier to trace; coupling to orchestrator.
- **At-Least-Once** → Message delivered ≥1 time; consumer must be idempotent.
- **Exactly-Once** → Hardest guarantee; Kafka transactions + idempotent producer; high overhead.

Patterns

- **Strangler Fig** → Wrap legacy with new service; migrate traffic incrementally; never big-bang rewrite.
- **Anti-Corruption Layer** → Translate between legacy and new domain model; isolates bounded contexts.
- **Sidecar** → Attach helper process (logging, service mesh proxy) to main service container.
- **Ambassador** → Proxy that handles network concerns (retry, circuit break) on behalf of main service.

Interview Deep-Dive Keywords

- "Microservices distributed complexity: you trade deployment independence for distributed systems problems"
- "Choreography scales better than orchestration but is harder to debug — use distributed tracing"
- "Exactly-once is a myth in most systems — design for idempotency and at-least-once delivery"
- "API Gateway is the front door — don't bypass it; put auth, rate limiting, logging there"

---

12. AWS Core Concepts & Infrastructure

Global Infrastructure

- **Regions** → 30+ geographic areas; each isolated; choose based on latency, compliance, service availability.
- **Availability Zones** → 2–6 per region; physically separate; connected by low-latency fiber; HA building block.
- **Local Zones** → AWS infrastructure in metro areas; low-latency for gaming, media, ML inference.
- **Wavelength Zones** → AWS compute embedded in telecom 5G networks; ultra-low latency for mobile edge.
- **AWS Outposts** → AWS rack installed on-prem; run AWS services locally; hybrid cloud.

- **Edge Locations / PoPs** → 400+ CloudFront edge locations; Route 53 anycast; WAF at edge.

Account & Organization

- **AWS Organizations** → Centrally manage multiple AWS accounts; Service Control Policies (SCPs); consolidated billing.
- **SCP (Service Control Policy)** → Maximum permission guardrails at OU/account level; overrides IAM; cannot grant, only restrict.
- **OU (Organizational Unit)** → Group accounts; apply SCPs; structure: Root → OU → Account.
- **Control Tower** → Automated multi-account setup; landing zones; guardrails; account vending.
- **Resource Tagging** → Key-value metadata on resources; mandatory for cost allocation, automation, security.

IAM

- **IAM User** → Long-term credentials; use for humans only when necessary; prefer roles.
- **IAM Role** → Assumed temporarily; for services (EC2, Lambda), cross-account, federated users.
- **IAM Policy** → JSON document; Allow/Deny Effect + Action + Resource + Condition.
- **Permission Boundary** → Limits maximum permissions a role/user can have; delegates safely.
- **IAM Identity Center (SSO)** → Centralized access to multiple AWS accounts; integrates with IdP (Okta, AD).
- **Condition Keys** → `aws:SourceVpc`, `s3:prefix`, `aws:MultiFactorAuthPresent`; fine-grained control.
- **Policy Evaluation** → Explicit Deny > Allow; default Deny; SCP → Resource Policy → Identity Policy → Session Policy.
- **Keywords**: Role > User for everything non-human; never use root except to create first admin; SCPs are guardrails not grants.

Interview Deep-Dive Keywords

- "SCPs do not grant permissions — they set the maximum ceiling; IAM policies must still allow"
- "IAM policy evaluation: explicit deny always wins; then check allow; default is deny"
- "Permission boundaries are the safe way to delegate IAM administration without privilege escalation"
- "AWS Organizations + Control Tower = automated account vending + guardrails for enterprise"

13. AWS Compute

EC2

- **EC2** → Elastic Compute Cloud; resizable VMs; widest instance family selection.
- **Instance Families** → General (M, T), Compute (C), Memory (R, X), Storage (I, D), GPU (P, G), ARM Graviton (M7g, C7g).
- **T-series (Burstable)** → CPU credits; burst above baseline; `T3a/T4g`; good for variable workloads.
- **Placement Groups** → Cluster (low latency, same AZ HPC), Spread (separate hardware, max 7/AZ), Partition (isolated partitions, Cassandra/HDFS).
- **User Data** → Bootstrap script on first launch; install packages, configure agent; runs as root.
- **Instance Metadata Service (IMDS)** → `169.254.169.254`; get temp credentials, instance ID; use IMDSv2 (session-token) — IMDSv1 is SSRF risk.
- **AMI** → Amazon Machine Image; baked OS + config; immutable artifact for fleet deployment.
- **Spot Instances** → Unused capacity; up to 90% discount; 2-min interruption notice; stateless/batch workloads.
- **Dedicated Hosts** → Physical server reserved for you; licensing compliance (BYOL); most expensive.

Auto Scaling & Load Balancing

- **ASG (Auto Scaling Group)** → Define min/desired/max; launch template; health check replacement.
- **Launch Template** → Preferred over Launch Config; versioned; supports mixed instance policies.
- **ALB (Application Load Balancer)** → L7; HTTP/HTTPS/WebSocket; path/host/header routing; target groups.
- **NLB (Network Load Balancer)** → L4; TCP/UDP/TLS; static IP; ultra-low latency; millions rps.
- **GLB (Gateway Load Balancer)** → Route traffic through third-party appliances (firewalls, IDS/IPS); bump-in-the-wire.
- **Target Groups** → Collection of targets (EC2, Lambda, containers, IPs); LB routes to healthy targets.
- **Connection Draining** → Allow in-flight requests to complete before deregistering instance; `deregistration_delay`.

Lambda

- **Lambda** → Event-driven FaaS; up to 15 min; 10GB RAM; 512MB–10GB /tmp; 1000 concurrent default.
- **Execution Environment** → Firecracker microVM; reused across warm invocations; share `/tmp` across warm reuse.
- **Lambda Layers** → Share code/dependencies across functions; up to 5 layers; 250MB total.
- **Lambda@Edge** → Run Lambda at CloudFront edge; low-latency request/response manipulation; max 5s.
- **Event Source Mapping** → Polls SQS/Kinesis/DynamoDB Streams on your behalf; batch processing.
- **Destinations** → Route success/failure to SQS/SNS/EventBridge/another Lambda; async error handling.

Other Compute

- **Elastic Beanstalk** → PaaS; deploy app + managed infra; supports Java, Node, Python, Docker; less control.
- **ECS** → Managed container orchestration; EC2 launch type (you manage nodes) or Fargate (serverless).
- **EKS** → Managed Kubernetes; AWS manages control plane; worker nodes on EC2 or Fargate.
- **Fargate** → Serverless containers; no node management; per-pod billing; slower start than EC2.
- **App Runner** → Fully managed container service; auto-scale; built-in LB; simplest container deployment.
- **Batch** → Managed batch processing; dynamically provisions compute; job queues and definitions.

Interview Deep-Dive Keywords

- "Graviton instances (ARM) offer 20-40% better price-performance — use for new workloads"
- "IMDSv2 is mandatory — IMDSv1 is vulnerable to SSRF attacks; enforce via instance metadata options"
- "Lambda concurrency × DB connections = disaster; always use RDS Proxy or connection pool in front of RDS"
- "ASG health check: EC2 health check replaces terminated instances; ELB health check replaces unhealthy ones — use ELB health check type"

---

14. AWS Storage

S3

- **S3** → Object storage; unlimited scale; 11 nines durability; 99.99% availability; key-value (key = full path).
- **Storage Classes**:
  - `S3 Standard` → Hot data; frequent access; highest cost.
  - `S3 Intelligent-Tiering` → Auto-move between tiers based on access; small monitoring fee.
  - `S3 Standard-IA` → Infrequent access; lower storage cost; retrieval fee; min 30-day charge.
  - `S3 One Zone-IA` → Single AZ; 20% cheaper than Standard-IA; for non-critical replicated data.
  - `S3 Glacier Instant Retrieval` → Archives accessed quarterly; millisecond retrieval.
  - `S3 Glacier Flexible Retrieval` → Archives; minutes-to-hours retrieval; cheapest active archive.
  - `S3 Glacier Deep Archive` → Lowest cost; 12-hour retrieval; regulatory long-term retention.
- **S3 Lifecycle Policies** → Auto-transition objects between classes or expire; based on age or prefix.
- **S3 Versioning** → Preserve every version; protects against accidental delete; costs multiply.
- **S3 Replication** → CRR (cross-region), SRR (same-region); requires versioning enabled.
- **S3 Object Lock** → WORM (Write Once Read Many); compliance or governance mode; protect from deletion.
- **S3 Event Notifications** → Trigger Lambda/SQS/SNS on object create/delete; event-driven pipelines.
- **S3 Transfer Acceleration** → Uploads via CloudFront edge → AWS backbone; faster for distant uploaders.
- **Presigned URLs** → Time-limited URL to GET/PUT a private object; signed with IAM credentials.
- **S3 Access Points** → Named network-specific access policies; simplify access control at scale.

Block & File Storage

- **EBS (Elastic Block Store)** → Network-attached block storage; persists beyond instance lifetime; single AZ; one attachment (except Multi-Attach io2).

- `gp3` → Default SSD; IOPS/throughput independently configurable; better than gp2 for most workloads.
- `io2 Block Express` → High-performance; 64K+ IOPS; databases; expensive.
- `st1` → Throughput-optimized HDD; sequential big data; log processing.
- `sc1` → Cold HDD; infrequently accessed; cheapest block storage.

- **EFS (Elastic File System)** → Managed NFS; multi-AZ; auto-scales; Linux only; 3× EBS cost; shared access.
- **FSx for Windows** → Managed SMB/NTFS; Windows workloads; Active Directory integration.
- **FSx for Lustre** → High-performance parallel FS; ML training, HPC; integrates with S3.
- **AWS Backup** → Centralized backup across EBS, RDS, DynamoDB, EFS, FSx; lifecycle policies; cross-region.

Interview Deep-Dive Keywords

- "S3 is not a file system — key-value object store; no directories (prefix simulation only)"
- "gp3 is always better than gp2 — cheaper, independently configurable IOPS; migrate now"
- "EBS = one AZ, one instance; EFS = multi-AZ, many instances; choose based on sharing requirement"
- "S3 Object Lock for compliance retention — nothing can delete it, not even root; use with Glacier Deep Archive"

---

15. AWS Networking

VPC Deep Dive

- **VPC** → Isolated virtual network; spans all AZs in a region; /16 to /28 CIDR.
- **Subnet** → Tied to one AZ; public (IGW route) or private; CIDR must not overlap with other subnets.
- **Internet Gateway** → Allows public subnet traffic to internet; 1:1 NAT for Elastic IPs; fully managed.
- **NAT Gateway** → Managed; in public subnet; private subnets route outbound traffic through it; ~$0.045/hr + data.
- **VPC Endpoints**:
  - `Gateway Endpoint` → S3 and DynamoDB; free; route table entry; private access.
  - `Interface Endpoint (PrivateLink)` → All other services; ENI with private IP; per-AZ; charges apply.

- **Security Group** → Instance-level stateful firewall; allow-only; reference other SGs by ID.
- **NACL** → Subnet-level stateless; ordered rules; explicit allow + deny; applied before SGs.
- **Flow Logs** → Capture IP traffic info for VPC/subnet/ENI; publish to CloudWatch or S3; security forensics.

Cross-VPC & Hybrid

- **VPC Peering** → Direct private routing; no transitive routing; 1:1 relationship; requires non-overlapping CIDRs.
- **Transit Gateway (TGW)** → Regional hub; connect VPCs + on-prem via single gateway; transitive routing; route tables.
- **AWS PrivateLink** → Expose service to other VPCs without peering; direction-aware; no routing complexity.
- **Site-to-Site VPN** → IPSec tunnel; two tunnels for redundancy; BGP or static; up to 1.25 Gbps per tunnel.
- **Direct Connect** → Dedicated 1/10/100 Gbps circuit; consistent bandwidth; lower latency; 3 months to provision.
- **Direct Connect Gateway** → Connect one Direct Connect to multiple VPCs across regions.

DNS & CDN

- **Route 53** → Authoritative DNS; health checks; routing policies: Simple, Weighted, Latency, Failover, Geolocation, Geoproximity, Multi-value.
- **Route 53 Resolver** → Hybrid DNS; Forward rules (VPC → on-prem), Inbound endpoints (on-prem → VPC).
- **CloudFront** → CDN; 400+ PoPs; cache TTL; Lambda@Edge; signed cookies/URLs; Origin Shield.
- **CloudFront Functions** → Lighter than Lambda@Edge; sub-millisecond; request/response header manipulation only.
- **AWS Global Accelerator** → Anycast static IPs; route to nearest healthy endpoint over AWS backbone; not CDN — improves TCP/UDP.

Interview Deep-Dive Keywords

- "Security groups are stateful — if you allow inbound port 80, return traffic is automatically allowed"
- "NACLs are stateless — you must allow both inbound AND outbound for a connection to work"
- "Transit Gateway solves the VPC peering mesh problem — N VPCs = N TGW attachments, not N*(N-1)/2 peerings"

- "Direct Connect ≠ VPN — Direct Connect is a dedicated physical circuit; VPN is encrypted tunnel over internet"

---

16. AWS Databases

Relational

- **RDS** → Managed RDBMS; MySQL, PostgreSQL, MariaDB, Oracle, SQL Server; Multi-AZ for HA; Read Replicas for scale.
- **RDS Multi-AZ** → Synchronous standby in another AZ; automatic failover (~1 min); not for read scaling.
- **RDS Read Replica** → Asynchronous replication; read scaling; can promote to standalone; cross-region supported.
- **RDS Proxy** → Connection pooler; reduces connection overhead; faster failover; mandatory for Lambda + RDS.
- **Aurora** → MySQL/PostgreSQL compatible; shared storage across 6 copies in 3 AZs; 3-5× faster than MySQL.
- **Aurora Serverless v2** → Auto-scale compute in fractions of ACU; pay per second; good for variable workloads.
- **Aurora Global Database** → Cross-region replication; <1s lag; promote secondary for DR; ~60-second RTO.
- **RDS Automated Backups** → Daily snapshot + transaction logs; PITR up to 35 days; stored in S3.

NoSQL

- **DynamoDB** → Managed KV + document; single-digit ms at any scale; serverless capacity or provisioned.
- **DynamoDB On-Demand** → Pay per request; no capacity planning; higher per-request cost; best for unpredictable traffic.
- **DynamoDB Provisioned + Auto Scaling** → Define RCU/WCU; cheaper for predictable traffic.
- **DynamoDB DAX** → In-memory cache for DynamoDB; microsecond reads; write-through; DynamoDB API compatible.
- **DynamoDB Global Tables** → Multi-region active-active; eventually consistent replication; conflict resolution via last-write-wins.
- **DynamoDB Streams** → Change data capture; event-driven processing; Lambda trigger; 24-hour retention.

- **Single-Table Design** → Overload one table with multiple entity types; pk/sk access patterns; efficient but complex.

Analytics & Cache

- **Redshift** → Managed data warehouse; columnar; MPP; RA3 nodes separate storage/compute; Redshift Spectrum queries S3.
- **ElastiCache Redis** → Managed Redis; cluster mode; multi-AZ; persistence; pub/sub; Lua scripts.
- **ElastiCache Memcached** → Managed Memcached; simpler; no persistence; multi-threaded; pure cache.
- **MemoryDB for Redis** → Redis-compatible; durable; multi-AZ; primary DB use case (not just cache).
- **Neptune** → Managed graph DB; Property Graph (Gremlin) + RDF (SPARQL); ACID.
- **Keyspaces** → Managed Apache Cassandra; serverless; CQL compatible.
- **Timestream** → Managed time-series DB; auto-tiering (memory → magnetic); SQL-like query.

Interview Deep-Dive Keywords

- "Aurora storage is shared and replicated 6 ways — you pay for storage + compute separately; much more resilient than RDS"
- "DynamoDB single-table design: understand access patterns first, then design pk/sk — not the other way"
- "RDS Multi-AZ is for HA/failover; Read Replica is for read scale — they solve different problems"
- "DAX is a write-through cache — writes go to DynamoDB first; use for read-heavy, latency-sensitive workloads"

---

17. AWS Security

IAM Advanced

- **AWS IAM** → Identity and Access Management; global service; controls all AWS API calls.
- **Resource-Based Policies** → Attached to resources (S3 bucket, KMS key, Lambda); define who can access from outside account.
- **Identity-Based Policies** → Attached to IAM identity; managed (AWS/customer) or inline.
- **Role Trust Policy** → Defines who can assume the role (`sts:AssumeRole`); principal = user, service, account.

- **AWS STS** → Security Token Service; issues temporary credentials on role assumption; credentials expire.
- **IAM Access Analyzer** → Detect resources shared externally; validate policies; generate least-privilege policies from CloudTrail.

Key Management & Secrets

- **KMS (Key Management Service)** → Create/manage/rotate encryption keys; hardware security modules; CMK (Customer Managed Key).
- **CMK (Customer Managed Key)** → You control key policy + rotation; audit key usage in CloudTrail.
- **AWS Managed Keys** → AWS creates/rotates for each service; you can't see key material.
- **Envelope Encryption** → Data encrypted with data key (DEK); DEK encrypted with KMS CMK; DEK stored with data.
- **Secrets Manager** → Store/rotate/retrieve secrets; auto-rotation via Lambda; charge per secret.
- **Parameter Store** → Store config + secrets; free tier for standard; SecureString = KMS-encrypted; no auto-rotation.

Threat Detection & Protection

- **GuardDuty** → Managed threat detection; analyzes CloudTrail, VPC Flow Logs, DNS logs; ML-based anomaly detection; no agent needed.
- **Inspector** → Automated vulnerability scanning; EC2 (OS/packages), ECR images, Lambda functions; CVE-based.
- **Macie** → ML-based PII discovery in S3; classify and protect sensitive data.
- **Security Hub** → Aggregated security findings; normalized; integrates GuardDuty, Inspector, Macie, Config.
- **WAF** → Web Application Firewall; L7 protection; OWASP rules; rate limiting; attach to ALB/CloudFront/API GW.
- **Shield Standard** → Automatic DDoS protection; free; L3/L4; always on.
- **Shield Advanced** → $3000/month; L7 DDoS; 24/7 DDoS Response Team; cost protection.
- **Network Firewall** → Managed stateful firewall for VPC; IDS/IPS; Suricata-compatible rules.

Interview Deep-Dive Keywords

- "GuardDuty should be enabled in every account, every region — it's passive, low-cost, high-value"
- "KMS envelope encryption: never encrypt large data directly with KMS — encrypt DEK, encrypt data with DEK"

- "Secrets Manager > Parameter Store for secrets needing rotation; Parameter Store for config values"
- "IAM Access Analyzer = automated external sharing audit — run weekly and review findings"

---

18. AWS Monitoring & DevOps

Monitoring

- **CloudWatch Metrics** → Time-series metrics; EC2/RDS/Lambda built-in; custom metrics via PutMetricData API.
- **CloudWatch Logs** → Aggregate logs; Log Groups + Log Streams; Logs Insights for SQL-like queries.
- **CloudWatch Alarms** → Threshold + anomaly detection; trigger SNS/Auto Scaling/EC2 action.
- **CloudWatch Dashboards** → Visualize metrics cross-region; share with stakeholders.
- **CloudWatch Container Insights** → ECS/EKS metrics + logs; cluster/service/task level.
- **CloudWatch Lambda Insights** → Enhanced Lambda metrics; memory, init duration, cold start tracking.
- **CloudTrail** → API audit log; every AWS API call; management events (free) + data events (charged); immutable.
- **AWS Config** → Resource configuration history; compliance rules; drift detection; remediation actions.
- **EventBridge** → Serverless event bus; route events from AWS services, SaaS, custom apps; schedule with cron.
- **X-Ray** → Distributed tracing; service map; latency analysis; identify bottlenecks in distributed apps.

Infrastructure as Code

- **CloudFormation** → AWS-native IaC; JSON/YAML templates; stacks; change sets preview before apply.
- **CDK (Cloud Development Kit)** → Define infra in Python/TypeScript/Java; synthesizes CloudFormation; higher-level constructs.
- **Terraform** → Multi-cloud IaC; HCL; state file; plan/apply workflow; large ecosystem.
- **AWS SAM** → Serverless Application Model; CloudFormation extension; simplified Lambda/API GW/DynamoDB syntax.

- **Change Sets** → Preview CloudFormation changes before applying; mandatory for production changes.
- **Stack Drift** → Detect manual changes to CloudFormation-managed resources; drift detection report.

CI/CD

- **CodeCommit** → Managed Git; being deprecated in favor of third-party (GitHub, GitLab).
- **CodeBuild** → Managed build service; Docker-based; pay per minute; run tests, build images.
- **CodeDeploy** → Automated deployment to EC2, Lambda, ECS; blue/green and rolling strategies.
- **CodePipeline** → Orchestrate CI/CD pipeline; source → build → test → deploy; integrates third-party tools.
- **ECR** → Elastic Container Registry; private Docker registry; image scanning; lifecycle policies.

Interview Deep-Dive Keywords

- "CloudTrail is your audit log — enable in all regions, all accounts, log to S3 with MFA delete"
- "CloudFormation change sets are mandatory for production — never apply without reviewing what changes"
- "CDK vs Terraform: CDK = AWS-native, imperative; Terraform = multi-cloud, declarative; use Terraform for multi-cloud"
- "EventBridge replaces CloudWatch Events — richer routing, schema registry, SaaS integrations"

---

19. AWS Advanced Patterns

Multi-Account Architecture

- **Account per Environment** → Dev/Staging/Prod in separate accounts; blast radius containment.
- **Account per Team/Service** → Large orgs; team owns account; limit cross-team blast radius.
- **Shared Services Account** → Central logging, security tooling, DNS, Transit Gateway hub.
- **Log Archive Account** → Immutable CloudTrail + Config log destination; read-only for all.
- **Security Account** → GuardDuty master, Security Hub aggregator, IAM Identity Center.
- **AWS Landing Zone** → Baseline multi-account setup; Control Tower automates this.

Serverless Patterns

- **EventBridge → Step Functions → Lambda** → Event-driven orchestrated workflow.
- **SQS → Lambda** → Async event processing; built-in retry + DLQ; scale Lambda to match queue depth.
- **API Gateway → Lambda → DynamoDB** → Classic serverless web API stack.
- **S3 → Lambda → S3** → Object processing pipeline (resize image, parse CSV, scan file).
- **Kinesis Data Streams → Lambda** → Real-time stream processing; ordered within shard.

Data Architecture

- **Data Lake on S3** → Raw (bronze) / Cleaned (silver) / Aggregated (gold) zones; Parquet/ORC format.
- **AWS Glue** → Serverless ETL; data catalog; crawlers; job authoring in Python/Scala.
- **Athena** → Serverless SQL on S3; pay per query; Parquet/ORC dramatically reduce cost.
- **Lake Formation** → Centralized data lake governance; column/row-level security; fine-grained access.
- **Kinesis Data Firehose** → Managed streaming delivery to S3/Redshift/OpenSearch; no code; auto-scale.
- **EMR** → Managed Hadoop/Spark; big data processing; transient clusters for cost; Spot nodes for workers.

Interview Deep-Dive Keywords

- "Multi-account = blast radius containment; separate prod from dev at account level, not just IAM"
- "Athena + S3 + Parquet = cost-effective analytics without a data warehouse for most use cases"
- "Step Functions for orchestration with retries/catches/parallel states — don't build workflow logic in Lambda"
- "Kinesis = ordered, real-time streaming; SQS = at-least-once, high-volume queuing; they solve different problems"

---

20. Azure Core Concepts & Infrastructure

Global Infrastructure

- **Regions** → 60+ regions; paired regions for DR (East US paired with West US); choose based on latency + compliance.

- **Availability Zones** → 3 per region (where supported); fault-isolated; power/network/cooling independent.
- **Region Pairs** → Azure replicates some services automatically; platform updates rolled out one pair at a time.
- **Sovereign Clouds** → Azure Government (US), Azure China (21Vianet); physically isolated; separate portals.

Resource Hierarchy

- **Management Groups** → Top-level; group subscriptions; apply policies + RBAC; up to 6 levels deep.
- **Subscriptions** → Billing + access boundary; trust one Azure AD tenant; resource limits per subscription.
- **Resource Groups** → Logical container for resources; same lifecycle; same region recommended for management plane.
- **Resources** → Individual service instances (VM, DB, VNet); belong to one resource group.
- **Azure Resource Manager (ARM)** → Deployment and management layer; all Azure API calls go through ARM.

Governance

- **Azure Policy** → Enforce rules on resources; audit, deny, or auto-remediate; JSON policy definitions.
- **Initiative (Policy Set)** → Group of policies; assign together; CIS Benchmark, regulatory compliance initiatives.
- **RBAC** → Role assignments; built-in roles (Owner, Contributor, Reader); custom roles; scope: MG → Sub → RG → Resource.
- **Azure Blueprints** → Repeatable environment setup; combine RBAC + policies + ARM templates; for landing zones.
- **Azure AD (Entra ID)** → Cloud identity provider; users/groups/apps; B2B (external users), B2C (customer identity).

Interview Deep-Dive Keywords

- "Resource groups are for lifecycle management — put resources together if they're deployed/deleted together"
- "Azure Policy vs RBAC: Policy controls what can be done to resources; RBAC controls who can do it"
- "Subscriptions are the billing and limits boundary — split by environment or team at this level"

- "Management Groups are for multi-subscription governance — apply policies and RBAC at scale"

---

21. Azure Compute

Virtual Machines

- **Azure VMs** → IaaS; Windows + Linux; hundreds of SKUs; VM families: General (Dsv5), Compute (Fsv2), Memory (Esv5), GPU (NCv3).
- **VM Scale Sets (VMSS)** → Auto-scale group of identical VMs; Flexible or Uniform orchestration; rolling upgrades.
- **Azure Spot VMs** → Unused capacity; 60–90% discount; evicted with 30-second notice; stateless/batch only.
- **Reserved VM Instances** → 1 or 3-year commitment; up to 72% discount.
- **Dedicated Hosts** → Single-tenant physical server; BYOL compliance; control maintenance windows.
- **Proximity Placement Groups** → Co-locate VMs in same data center; lowest latency for HPC/distributed apps.

Platform Services

- **App Service** → Managed PaaS; deploy web apps, APIs, mobile backends; Windows + Linux; auto-scale; deployment slots.
- **App Service Plan** → Defines compute tier; shared across multiple apps; scale plan to scale all apps.
- **Deployment Slots** → Staging/production slots; swap with zero downtime; test in staging before promoting.
- **Azure Functions** → Serverless; Consumption (pay per exec), Premium (pre-warmed, VNet), Dedicated plans.
- **Durable Functions** → Stateful orchestration on Azure Functions; orchestrator + activity + entity functions.
- **Container Instances (ACI)** → Run containers without orchestration; per-second billing; dev/test/burst.
- **AKS (Azure Kubernetes Service)** → Managed Kubernetes; free control plane; VMSS node pools; Azure CNI / kubenet networking.

Interview Deep-Dive Keywords

- "App Service deployment slots enable zero-downtime deploys with easy rollback — use

them"

- "Durable Functions solve stateful serverless orchestration — like Step Functions for Azure"
- "AKS with Azure AD Workload Identity = pods get Azure AD identities; no secrets needed for Azure services"
- "VMSS Flexible orchestration allows mixing VM types — better for heterogeneous workloads than Uniform"

---

22. Azure Storage

Storage Types

- **Azure Blob Storage** → Object storage (like S3); Block blobs (files), Page blobs (VHD), Append blobs (logs).
- **Storage Tiers** → Hot (frequent), Cool (infrequent, 30-day min), Cold (rare, 90-day min), Archive (offline, 180-day min, hours rehydration).
- **Azure Files** → Managed SMB/NFS file shares; lift-and-shift for file servers; Azure File Sync for hybrid.
- **Azure Queue Storage** → Simple message queue; 64KB max per message; 7-day TTL; for decoupling.
- **Azure Table Storage** → NoSQL KV store; schemaless; cheap; superseded by Cosmos DB for most uses.
- **Azure Disk Storage** → Managed disks; Ultra (highest IOPS), Premium SSD v2, Premium SSD, Standard SSD, Standard HDD.

Data Protection

- **Soft Delete** → Retain deleted blobs/files for configurable period; restore accidental deletes.
- **Versioning** → Auto-save previous blob versions; retain history; point-in-time recovery.
- **Immutable Storage** → WORM policies; time-based or legal hold; regulatory compliance.
- **Geo-Redundancy Options** → LRS (3 copies, 1 DC), ZRS (3 AZs), GRS (LRS + async to paired region), GZRS (ZRS + async to paired region).
- **Azure Backup** → Centralized backup for VMs, databases, files; long-term retention; cross-region restore.

Interview Deep-Dive Keywords

- "LRS = cheapest, one data center failure risk; GRS = cross-region, recovery objective after regional disaster"

- "Archive tier requires rehydration (hours to days) — not suitable for data that might need quick restore"
- "Azure File Sync creates hybrid file system — edge caches hot files, cold files stay in Azure"
- "Premium SSD v2 = independently configure IOPS + throughput + capacity; ideal for DBs"

---

23. Azure Networking

Core Networking

- **Virtual Network (VNet)** → Isolated network; region-scoped; address space in CIDR; subnets within.
- **Network Security Group (NSG)** → L3/L4 stateful firewall; inbound + outbound rules; apply to NIC or subnet.
- **Application Security Group (ASG)** → Group VMs logically; reference ASG in NSG rules instead of IPs; cleaner rules.
- **User Defined Routes (UDR)** → Override default routing; force traffic through NVA/firewall.
- **Azure Firewall** → Managed stateful firewall; FQDN rules; threat intelligence; east-west + north-south.
- **DDoS Protection** → Basic (free, always on); Network (enhanced, SLA, cost protection for plan resources).
- **Private Endpoint** → Private IP for Azure PaaS (Storage, SQL, Key Vault) in your VNet; eliminates public internet path.
- **Service Endpoint** → Route traffic to Azure services over Azure backbone; simpler than Private Endpoint; still uses public IP.

Connectivity

- **VNet Peering** → Low-latency private connectivity; not transitive; global peering across regions.
- **Azure Virtual WAN** → Microsoft-managed hub-and-spoke; global transit; replaces Transit Gateway equivalent.
- **VPN Gateway** → IPSec/IKE tunnels; up to 10 Gbps (VpnGw5); active-active for HA.
- **ExpressRoute** → Dedicated private circuit; 50 Mbps–100 Gbps; Global Reach for cross-circuit routing.
- **Azure DNS** → Managed DNS; private DNS zones for VNet resolution; auto-registration.
- **Azure Front Door** → Global HTTP/S load balancer + CDN + WAF; route to nearest healthy backend; SSL offload.

- **Traffic Manager** → DNS-based global traffic routing; Performance, Priority, Weighted, Geographic methods; no data path.
- **Application Gateway** → Regional L7 LB; WAF; SSL termination; URL-based routing; autoscale.

Interview Deep-Dive Keywords

- "NSGs are stateful — allow inbound port 443, return traffic is automatic"
- "Private Endpoint > Service Endpoint for security — Private Endpoint keeps traffic completely off internet"
- "Azure Front Door for global HTTP apps; Traffic Manager for DNS-level global routing of non-HTTP"
- "VNet peering is not transitive — use Azure Virtual WAN or NVA for hub-and-spoke with transit"

---

24. Azure Databases

Relational

- **Azure SQL Database** → PaaS SQL Server; serverless or provisioned; DTU or vCore models; hyperscale option.
- **Azure SQL Managed Instance** → Near 100% SQL Server compatibility; VNet-native; lift-and-shift from on-prem SQL.
- **Azure DB for PostgreSQL** → Flexible Server (preferred); zone-redundant HA; read replicas; compute auto-pause.
- **Azure DB for MySQL** → Flexible Server; zone-redundant HA; read replicas.
- **SQL Elastic Pool** → Share compute across multiple databases; cost-effective for variable SaaS DBs.
- **Active Geo-Replication** → SQL Database feature; up to 4 readable secondaries; cross-region.
- **Auto-Failover Groups** → Abstraction over geo-replication; read/write endpoint auto-switches on failover.

NoSQL & Analytics

- **Cosmos DB** → Globally distributed multi-model DB; 5 consistency levels; 99.999% availability SLA; multiple APIs (SQL, MongoDB, Cassandra, Gremlin, Table).
- **Cosmos DB Consistency Levels** → Strong → Bounded Staleness → Session → Consistent Prefix → Eventual; tune per operation.

- **Cosmos DB Serverless** → Pay per RU; good for dev/variable workloads; no multi-region.
- **Cosmos DB Autoscale** → Set max RU; auto-scale 10%–100% of max; no manual capacity management.
- **Azure Synapse Analytics** → Unified analytics; SQL pools (dedicated DWH), Spark pools, Synapse Link (HTAP from Cosmos DB).
- **Azure Cache for Redis** → Managed Redis; C and P tiers; zone redundancy; geo-replication.

Interview Deep-Dive Keywords

- "Cosmos DB consistency: Strong = highest cost + latency; Session = default, read-your-writes guarantee"
- "Azure SQL Managed Instance = easiest on-prem SQL Server migration path; near-100% compat"
- "Synapse Link = zero-ETL analytics on Cosmos DB operational data; near-real-time HTAP"
- "Elastic Pool = cost optimization for many small databases with variable and non-overlapping peaks"

---

25. Azure Security

Identity

- **Azure AD (Entra ID)** → Cloud IAM; every Azure resource auth goes through it; Conditional Access; PIM.
- **Conditional Access** → Policy-based access: location + device + risk + MFA; enforced at login.
- **PIM (Privileged Identity Management)** → Just-in-time elevated access; approve/time-limit admin roles; audit trail.
- **Managed Identities** → System-assigned (per resource) or user-assigned (shared); no credentials; auto-rotated; use for all Azure service auth.
- **Service Principals** → App identity; client ID + secret or certificate; manual rotation; less preferred than Managed Identity.
- **Azure AD B2B** → Invite external users to your Azure AD tenant; guest access.
- **Azure AD B2C** → Customer identity platform; social login (Google, Facebook); custom flows; millions of users.

Data & Key Protection

- **Azure Key Vault** → Secrets, keys, certificates; HSM-backed tiers; RBAC + access policies; soft-delete + purge protection.

- **Defender for Cloud** → CSPM + CWPP; security posture score; recommendations; threat protection across hybrid.
- **Microsoft Sentinel** → Cloud-native SIEM + SOAR; log analytics; ML-based threat detection; automated response.
- **Azure DDoS Protection** → Network-level protection; adaptive tuning; cost protection guarantee.
- **Azure Policy + Defender** → Enforce encryption, private endpoints, no public IPs; auto-remediation.

Interview Deep-Dive Keywords

- "Managed Identity is the Azure equivalent of IAM Roles for EC2 — zero-secret, auto-rotated auth"
- "PIM for all privileged roles — just-in-time, not just-in-case; require justification + approval"
- "Conditional Access is the Zero Trust policy engine for Azure AD — enforce MFA, device compliance"
- "Key Vault purge protection + soft delete = protection against accidental or malicious key deletion"

---

26. Azure Monitoring & DevOps

Monitoring

- **Azure Monitor** → Central platform; metrics, logs, traces; integrates all Azure services.
- **Log Analytics Workspace** → Central log store; Kusto Query Language (KQL); cross-resource queries.
- **Application Insights** → APM; SDK-based; request traces, dependency tracking, exceptions, custom metrics.
- **Azure Monitor Metrics** → Numeric time-series; built-in for every resource; 93-day retention; free.
- **Alerts** → Metric alerts, log alerts, activity log alerts; action groups (email, SMS, webhook, ITSM).
- **Workbooks** → Interactive visualization combining metrics, logs, parameters; shareable dashboards.
- **Diagnostic Settings** → Route resource logs/metrics to Log Analytics, Event Hubs, Storage.

DevOps

- **Azure DevOps** → Boards (work items), Repos (Git), Pipelines (CI/CD), Test Plans, Artifacts.

- **Azure Pipelines** → YAML-based CI/CD; multi-stage; environment approvals; parallel jobs.
- **ARM Templates** → JSON-based IaC; declarative; complex syntax; complete mode (remove unlisted resources).
- **Bicep** → DSL compiling to ARM; cleaner syntax; no state file needed; IDE support; preferred over raw ARM.
- **Azure Container Registry (ACR)** → Private Docker registry; geo-replication; ACR Tasks for automated builds.
- **GitHub Actions + Azure** → OIDC federation for passwordless deployment; `azure/login` action.

Interview Deep-Dive Keywords

- "Bicep over ARM templates — same underlying model but dramatically better developer experience"
- "Application Insights sampling: enable adaptive sampling in production to control cost"
- "Log Analytics KQL: master `summarize`, `project`, `join`, `extend` — these cover 90% of queries"
- "Azure Pipelines environment approvals: require manual approval before deploying to production"

---

27. Azure Advanced Patterns

Hybrid & Multi-Cloud

- **Azure Arc** → Extend Azure management to on-prem, other clouds; Arc-enabled servers, Kubernetes, data services.
- **Azure Stack Hub** → Azure services in your own data center; disconnected scenarios; government.
- **Azure Stack Edge** → Edge compute device; AI inference at edge; Azure-managed; hardware appliance.

High Availability & DR

- **Availability Zones** → Spread VMs, managed disks, SQL DB across 3 zones; survive zone failure.
- **Paired Regions** → Automatic replication for some services; sequential platform updates; recommended DR target.
- **Traffic Manager + Front Door** → Global failover; Traffic Manager for DNS-level, Front Door for HTTP-level.

- **Azure Site Recovery (ASR)** → VM replication + orchestrated failover; on-prem to Azure or Azure to Azure DR.
- **Geo-Redundant Storage** → Data replicated to paired region; accessible only on failover (or RA-GRS: readable always).

Interview Deep-Dive Keywords

- "Azure Arc is Azure control plane for anything — manage on-prem servers and EKS clusters from Azure portal"
- "Site Recovery for VM DR: RPO minutes, RTO 1-2 hours; test failover without impacting production"
- "Paired regions get sequential platform updates — you have time to validate one region before the other updates"

---

28. GCP Core Concepts & Infrastructure

Global Infrastructure

- **Regions** → 40+ regions; choose for latency, compliance, service availability.
- **Zones** → 3+ per region; equivalent to AZs; failure-independent.
- **Multi-Region** → Some services (GCS, Spanner, BigQuery) operate multi-region natively; higher durability/availability.
- **Premium vs Standard Network Tier** → Premium: Google's private backbone (faster, lower latency); Standard: public internet routing (cheaper).
- **Points of Presence (PoPs)** → Cloud CDN and Cloud Interconnect endpoints; 150+ worldwide.

Resource Hierarchy

- **Organization** → Root node; tied to Google Workspace/Cloud Identity domain; policies here apply everywhere.
- **Folders** → Group projects; department/team level; inherit policies.
- **Projects** → Billing + quota + API boundary; resources belong to a project; equivalent to AWS account (roughly).
- **Resources** → Compute, storage, databases; belong to a project.

IAM

- **IAM Roles** → Basic (Owner/Editor/Viewer — avoid), Predefined (fine-grained per service), Custom (user-defined).

- **Service Accounts** → VM/app identity; key-based or workload identity; avoid downloading keys.
- **Workload Identity Federation** → Allow external identities (GitHub Actions, AWS, Azure) to impersonate service accounts; no key files.
- **IAM Conditions** → Attribute-based access; time-based access windows; resource tag conditions.
- **Policy Inheritance** → Policies at higher levels inherit down; no deny — use org policy constraints instead.
- **Org Policy Constraints** → Restrict resource configs (no public IPs, allowed regions, disable SA key creation).

Interview Deep-Dive Keywords

- "GCP IAM: avoid basic roles (Editor = too broad); use predefined or custom roles"
- "Workload Identity Federation = keyless auth; GitHub Actions deploy to GCP without service account JSON keys"
- "Org Policy constraints are deny-style guardrails — complement IAM which is allow-based"
- "Premium network tier routes traffic on Google backbone — measurable latency improvement globally"

---

29. GCP Compute

Compute Engine

- **Compute Engine** → GCP IaaS VMs; custom machine types (choose exact vCPU + RAM); live migration during host maintenance.
- **Machine Families** → General (N-series, E2), Compute (C3), Memory (M3), Accelerator (A3 GPU), Arm (T2A).
- **Preemptible VMs** → 80% discount; 30-second notice; 24-hour max lifetime; use for batch/fault-tolerant.
- **Spot VMs** → Like preemptible but no 24-hour limit; same pricing; preferred over preemptible for new workloads.
- **Committed Use Discounts (CUD)** → 1 or 3-year resource commitment; 37–57% discount; applies automatically.
- **Sustained Use Discounts (SUD)** → Automatic discounts for running VMs >25% of a month; no commitment.
- **Managed Instance Groups (MIG)** → Auto-scale group; rolling updates; health check replacement; equivalent to ASG.

Serverless & PaaS

- **Cloud Functions** → FaaS; event-driven; HTTP or background triggers; Gen2 on Cloud Run; up to 60 min.
- **Cloud Run** → Managed containers; scale to zero; HTTP-based; any language/framework; built on Knative.
- **Cloud Run Jobs** → Batch container execution; not HTTP-triggered; run to completion.
- **App Engine** → PaaS; Standard (language-specific sandboxes, fast scale) or Flexible (Docker-based, slower scale).
- **GKE (Google Kubernetes Engine)** → Most mature managed Kubernetes; Autopilot (fully managed nodes) or Standard (you manage nodes).
- **GKE Autopilot** → Google manages nodes, security hardening, bin packing; pay per pod resource; recommended for most.

Interview Deep-Dive Keywords

- "Cloud Run = easiest containerized app deployment on GCP; scales to zero; no Kubernetes knowledge needed"
- "GKE Autopilot vs Standard: Autopilot = less ops, higher per-pod cost; Standard = more control, more ops"
- "Custom machine types avoid over/under-provisioning — choose exact vCPU/RAM ratio for your workload"
- "Sustained Use Discounts are automatic — no commitment required; run VMs full month = 30% discount"

---

30. GCP Storage & Databases

Cloud Storage

- **Cloud Storage (GCS)** → Object storage; globally available; 11 nines durability; 4 storage classes.
  - `Standard` → Hot; frequent access.
  - `Nearline` → Infrequent; 30-day min; once/month access.
  - `Coldline` → Rare; 90-day min; once/quarter access.
  - `Archive` → Longest-term; 365-day min; retrieval fee; regulatory.
- **Object Versioning** → Preserve previous versions; lifecycle rules to delete old versions.
- **Retention Policies + Locks** → WORM; lock bucket to prevent deletion; compliance.

- **Signed URLs** → Time-limited access to private objects; equivalent to S3 presigned URLs.
- **Lifecycle Management** → Auto-transition or delete based on age, version count, storage class.

Databases

- **Cloud SQL** → Managed MySQL, PostgreSQL, SQL Server; HA with failover replica; read replicas; automated backups.
- **Cloud Spanner** → Globally distributed SQL; strong consistency; TrueTime; 99.999% SLA; horizontal scale; expensive.
- **BigQuery** → Serverless data warehouse; columnar; petabyte-scale; pay per query or flat rate; ML built-in.
- **Firestore** → Serverless document DB; real-time sync; mobile/web SDK; strong consistency; native or Datastore mode.
- **Bigtable** → Managed wide-column store; HBase-compatible; petabyte-scale; low-latency reads; IoT/time-series.
- **Memorystore** → Managed Redis or Memcached; VPC-native; in-region; equivalent to ElastiCache.
- **AlloyDB** → PostgreSQL-compatible; columnar engine for analytics; 4× faster than Cloud SQL for analytics.

Interview Deep-Dive Keywords

- "BigQuery = no infrastructure, no index, no cluster — just write SQL against petabytes; pay per TB scanned"
- "Cloud Spanner = relational + horizontal scale + global consistency — at a price; use for financial-grade global apps"
- "Firestore real-time listeners enable live data sync without polling — great for collaborative apps"
- "AlloyDB = the answer to 'PostgreSQL that can do OLTP and OLAP' on GCP"

---

31. GCP Networking & Security

Networking

- **VPC** → GCP VPC is global (spans regions); subnets are regional; simpler than AWS per-region VPCs.
- **Shared VPC** → Share VPC across projects; host project + service projects; centralized network management.

- **VPC Peering** → Connect VPCs; non-transitive; no overlapping subnets.
- **Cloud NAT** → Managed NAT; private instances outbound internet; no bastion needed for NAT.
- **Cloud Armor** → WAF + DDoS; OWASP rules; adaptive protection; attach to external HTTP LB.
- **Private Google Access** → VM instances without external IPs access Google APIs via private.googleapis.com.
- **VPC Service Controls** → Security perimeters around GCP services; prevent data exfiltration; context-aware.

Load Balancing

- **Global External HTTP(S) LB** → Anycast; global; Cloud CDN integration; URL map routing; premium tier.
- **Regional External HTTP(S) LB** → Regional; standard tier; simpler; lower cost.
- **TCP/SSL Proxy LB** → Global L4 TCP; terminates at Google edge.
- **Internal LB** → L4/L7; within VPC; for services that shouldn't be internet-accessible.
- **Cloud CDN** → Cache at Google PoPs; attached to global HTTP LB; signed URLs; cache invalidation.

Security

- **Cloud KMS** → Manage encryption keys; HSM option (Cloud HSM); CMEK for service encryption.
- **Secret Manager** → Store secrets; versioning; auto-rotation via Cloud Functions; access via IAM.
- **Security Command Center** → Centralized security findings; asset inventory; vulnerability scanning; threat detection.
- **Binary Authorization** → Require signed container images before GKE deployment; supply chain security.
- **VPC Service Controls** → Data exfiltration prevention; define service perimeters; restrict resource access by context.

Interview Deep-Dive Keywords

- "GCP VPC is global — one VPC, subnets per region; unlike AWS where VPC is per-region"
- "VPC Service Controls = the strongest data exfiltration control in GCP; essential for regulated data"
- "Cloud Armor adaptive protection auto-generates WAF rules from ML-detected attack patterns"

- "Binary Authorization enforces image signing — only images signed by approved authority deploy to GKE"

---

32. GCP Monitoring & DevOps

Monitoring

- **Cloud Monitoring** → Metrics ingestion; dashboards; alerting; 1500+ built-in metrics; custom metrics via API.
- **Cloud Logging** → Managed log aggregation; structured logs; Log Router to route to BigQuery/Pub/Sub/Storage.
- **Cloud Trace** → Distributed tracing; latency analysis; auto-instrumented for App Engine, Cloud Run.
- **Cloud Profiler** → Continuous production profiling; CPU + heap; low overhead; identify hot code paths.
- **Error Reporting** → Aggregate + count application errors; group similar errors; alert on new error types.
- **Uptime Checks** → External probe from multiple locations; alert on availability + latency.

DevOps & IaC

- **Cloud Build** → Managed CI/CD; `cloudbuild.yaml`; Docker builds; parallel steps; triggers from repo.
- **Artifact Registry** → Manage container images + language packages (Maven, npm, Python); CMEK; vulnerability scanning.
- **Deployment Manager** → GCP-native IaC; YAML/Python/Jinja; largely superseded by Terraform on GCP.
- **Terraform on GCP** → Preferred IaC; Google-maintained provider; large ecosystem.
- **Config Connector** → Manage GCP resources via Kubernetes CRDs; GitOps approach to GCP infra.
- **Cloud Deploy** → Managed continuous delivery; progression through environments; approval gates; rollback.

Interview Deep-Dive Keywords

- "Cloud Build + Artifact Registry + Cloud Deploy = complete GCP-native CI/CD pipeline"
- "Cloud Profiler with always-on, low-overhead profiling — no need to reproduce perf issues in staging"
- "Log Router to BigQuery enables log analytics at petabyte scale with SQL"

- "Terraform is preferred over Deployment Manager for GCP — better ecosystem and multi-cloud capability"

---

33. GCP Advanced Patterns

Data & Analytics Architecture

- **Pub/Sub** → Managed global messaging; push or pull; ordered delivery per key; Exactly-once with deduplication.
- **Dataflow** → Managed Apache Beam; streaming + batch unified; auto-scale; serverless; Pub/Sub → Dataflow → BigQuery.
- **Dataproc** → Managed Hadoop/Spark; fast cluster startup (~90 seconds); ephemeral clusters; Spot VMs for workers.
- **BigQuery ML** → Train ML models in SQL; no data movement; logistic regression, boosted trees, DNN.
- **Vertex AI** → Managed ML platform; training, serving, pipelines, feature store; MLOps.
- **Analytics Hub** → Share BigQuery datasets across organizations; data exchange marketplace.

Hybrid & Multi-Cloud

- **Anthos** → GKE anywhere (on-prem, AWS, Azure); manage fleet of clusters; Config Management (GitOps); Service Mesh.
- **Anthos Config Management** → GitOps; sync Kubernetes configs from Git across all clusters; policy controller.
- **Cross-Cloud Interconnect** → Dedicated physical connectivity between GCP and AWS/Azure; up to 100 Gbps.

Interview Deep-Dive Keywords

- "Pub/Sub + Dataflow + BigQuery = GCP's canonical real-time analytics pipeline"
- "Anthos = 'bring GCP to anywhere'; run GKE on-prem or in AWS; centralized policy management"
- "BigQuery partition + cluster: partition by date, cluster by high-cardinality column; reduces bytes scanned dramatically"
- "Vertex AI Feature Store solves the training/serving skew problem — same features for training and inference"

---

34. AWS vs Azure vs GCP Comparison

Compute Equivalents

Service	AWS	Azure	GCP
VMs	EC2	Virtual Machines	Compute Engine
Auto Scaling	Auto Scaling Groups	VM Scale Sets	Managed Instance Groups
FaaS	Lambda	Azure Functions	Cloud Functions / Cloud Run
Managed K8s	EKS	AKS	GKE
Serverless Containers	Fargate	Container Instances / ACA	Cloud Run
PaaS	Elastic Beanstalk	App Service	App Engine

Storage Equivalents

Service	AWS	Azure	GCP
Object Storage	S3	Blob Storage	Cloud Storage
Block Storage	EBS	Managed Disks	Persistent Disk
File Storage	EFS	Azure Files	Filestore
Archive	Glacier	Archive Tier	Archive class

Database Equivalents

Service	AWS	Azure	GCP
Managed PostgreSQL	RDS / Aurora	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
NoSQL Document	DynamoDB	Cosmos DB	Firestore
Data Warehouse	Redshift	Synapse Analytics	BigQuery
Global Distributed SQL	Aurora Global	Cosmos DB	Cloud Spanner
Cache	ElastiCache	Azure Cache for Redis	Memorystore

Networking Equivalents

Service	AWS	Azure	GCP
Virtual Network	VPC	VNet	VPC (global)
Private Connectivity	PrivateLink	Private Endpoint	Private Google Access
Dedicated Circuit	Direct Connect	ExpressRoute	Cloud Interconnect
Global LB	Global Accelerator	Front Door	Global HTTP(S) LB
CDN	CloudFront	Azure CDN / Front Door	Cloud CDN

Key Differentiators

- **AWS** → Widest service breadth; largest ecosystem; most mature; best for greenfield cloud-native.
- **Azure** → Best for Microsoft/enterprise shops; hybrid via Arc; Azure AD integration; Office 365 synergy.
- **GCP** → Best data/ML/analytics; global VPC simplicity; GKE maturity; BigQuery is unmatched for serverless analytics.
- **Keywords**: Choose cloud based on existing skills + ecosystem + workload type; not by marketing; enterprise = Azure, data/ML = GCP, greenfield = AWS.

---

35. Multi-Cloud & Hybrid Cloud

Multi-Cloud

- **Multi-Cloud** → Use multiple public cloud providers deliberately; avoid lock-in, best-of-breed services.
- **Cloud Agnostic Tools** → Terraform, Kubernetes, OpenTelemetry, Kafka; reduce provider-specific coupling.
- **Operational Complexity** → Multi-cloud multiplies operational overhead; teams need expertise in all; cost of coordination.
- **Data Gravity** → Data tends to stay where it is; egress costs and latency discourage multi-cloud for data-heavy workloads.
- **Vendor Lock-in Mitigation** → Containerize apps; use open standards; abstract cloud-specific APIs behind interfaces.

Hybrid Cloud

- **Hybrid Cloud** → On-prem + public cloud; connected securely; workloads split by sensitivity/performance/compliance.
- **AWS Outposts** → AWS rack on-prem; run EC2, RDS, EKS locally; consistent APIs; AWS manages hardware.
- **Azure Arc** → Project Azure management (Policy, RBAC, Defender) to on-prem and multi-cloud resources.
- **Google Anthos** → Run GKE on-prem (VMware, bare metal) or other clouds; centralized policy via Config Management.
- **VPN vs Direct Connect** → VPN = quick, cheap, variable latency; Direct Connect = consistent, dedicated, expensive.
- **Network Latency** → Synchronous cross-cloud calls add 10–50ms+; design async where possible.

Interview Deep-Dive Keywords

- "Multi-cloud ≠ resilience by default — apps must be architected for cross-cloud failover"
- "Terraform is the primary enabler of multi-cloud IaC — one tool, multiple providers"
- "Data gravity makes true multi-cloud hard — egress costs punish frequent cross-cloud data movement"
- "Hybrid = extend data center; multi-cloud = use multiple providers; common to have both simultaneously"

---

36. Cloud DevOps & CI/CD

CI/CD Principles

- **Continuous Integration (CI)** → Merge code frequently; automated build + test on every commit; fail fast.
- **Continuous Delivery (CD)** → Code always in deployable state; manual production gate acceptable.
- **Continuous Deployment** → Every passing commit auto-deploys to production; no manual gate.
- **Trunk-Based Development** → Short-lived feature branches; merge to main frequently; feature flags for incomplete work.
- **Feature Flags** → Toggle features without deploying; A/B testing; dark launches; gradual rollout.

- **GitOps** → Git as single source of truth for infra + app config; reconciliation loop deploys changes.

Deployment Strategies

- **Blue/Green** → Two identical environments; switch traffic at once; instant rollback; double the infra cost.
- **Canary** → Route small % of traffic to new version; gradually increase; auto-rollback on error spike.
- **Rolling** → Replace instances one-by-one; in-place; slower rollback; default for most managed services.
- **A/B Testing** → Route user segments to different versions; feature comparison; needs traffic splitting logic.
- **Shadow** → Duplicate production traffic to new version; no user impact; validate behavior before cutover.

Cloud CI/CD Tools

- **GitHub Actions** → Event-driven; OIDC-based cloud auth; matrix builds; reusable workflows; marketplace actions.
- **GitLab CI** → Integrated; `.gitlab-ci.yml`; runners (SaaS or self-hosted); environments + protected branches.
- **AWS CodePipeline + CodeBuild** → Native AWS; no external tooling; integrates with AWS services.
- **Azure Pipelines** → YAML multi-stage; environment approvals; parallel jobs; integrates with Azure services.
- **Cloud Build (GCP)** → Managed build execution; Cloud Deploy for delivery; Artifact Registry for artifacts.
- **Argo CD** → Kubernetes GitOps; reconcile cluster state to Git; self-healing; audit trail.
- **Flux** → CNCF GitOps operator; Helm + Kustomize support; multi-tenancy.

Interview Deep-Dive Keywords

- "Canary deployment is the safest production strategy — catch issues with 1% before 100%"
- "GitOps: if it's not in Git, it doesn't exist; Argo CD/Flux make Kubernetes GitOps production-ready"
- "OIDC-based auth for CI/CD: GitHub Actions → AWS/GCP/Azure without storing cloud credentials in secrets"
- "Blue/green needs 2x infra; canary needs traffic splitting; rolling needs zero-downtime health checks"

37. Cloud Observability & Monitoring

Three Pillars

- **Metrics** → Numeric time-series; counters, gauges, histograms; Prometheus, CloudWatch, Azure Monitor, Cloud Monitoring.
- **Logs** → Timestamped records; structured JSON preferred; CloudWatch Logs, Log Analytics, Cloud Logging.
- **Traces** → Request flow across services; spans + parent-child; X-Ray, Application Insights, Cloud Trace.

Standards & Tools

- **OpenTelemetry (OTel)** → Vendor-neutral SDK + collector; logs, metrics, traces; single instrumentation for multiple backends.
- **Prometheus** → Pull-based metrics; PromQL; cloud-native standard; scrapes /metrics endpoints.
- **Grafana** → Visualization; dashboards; multi-datasource; alerting; standard cloud observability UI.
- **Datadog** → Commercial APM + infra monitoring; deep AWS/Azure/GCP integrations; expensive but comprehensive.
- **New Relic** → Full-stack observability; NRQL; logs/metrics/traces unified.

Key Concepts

- **SLI / SLO / SLA** → SLI = measured indicator; SLO = target (99.9%); SLA = contract with penalty.
- **Error Budget** → 1 − SLO; burn rate alerting; reliability as product conversation.
- **RED Method** → Rate, Error rate, Duration; for service-level metrics.
- **USE Method** → Utilization, Saturation, Errors; for resource-level metrics.
- **4 Golden Signals** → Latency, Traffic, Errors, Saturation (Google SRE).
- **Distributed Tracing** → `traceId` propagated across all service calls; find root cause of latency.
- **Correlation ID** → Single request ID tied to all logs across all services; mandatory for microservices.

Interview Deep-Dive Keywords

- "OpenTelemetry is the future — instrument once, choose backend later; avoid vendor lock-

in for observability"

- "Alert on SLO burn rate, not raw metrics — SLO breach means user impact, CPU spike may not"
- "Structured logs (JSON) + correlation IDs + traceIds = the holy trinity for distributed system debugging"
- "Cardinality kills Prometheus — never use user_id or request_id as a label"

---

38. Cloud Cost Optimization

Framework

- **FinOps** → Financial accountability for cloud spend; shared responsibility across engineering + finance + product.
- **Tagging Strategy** → Tag every resource with `team`, `project`, `environment`, `cost-center`; non-tagged = unallocated waste.
- **Cost Allocation** → Attribute spend to teams/products; chargeback or showback model.
- **Budget Alerts** → Set thresholds; alert before overspend; AWS Budgets, Azure Cost Management, GCP Budgets.

Optimization Levers

- **Right-Sizing** → Match compute to actual utilization; AWS Compute Optimizer, Azure Advisor, GCP Recommender; most impactful action.
- **Reserved / Committed Use** → 1-3 year commitment for baseline; 30–72% savings; analyze steady-state usage first.
- **Spot / Preemptible for Batch** → 60–90% savings; fault-tolerant + stateless workloads only.
- **Auto Scaling** → Scale-in to zero when possible; stop paying for idle capacity.
- **Storage Lifecycle** → S3/Blob/GCS lifecycle rules; auto-tier cold data; delete expired data.
- **Data Transfer Optimization** → Minimize egress; keep compute and storage co-located; use CDN for static content.
- **Idle Resource Cleanup** → Unattached EBS, orphaned snapshots, unused LBs, idle RDS instances; schedule cleanup jobs.
- **Savings Plans (AWS)** → More flexible than RIs; commit to $/hr; covers Lambda + Fargate + EC2.

Interview Deep-Dive Keywords

- "The biggest cloud waste: resources running 24/7 that only need 8 hours; dev/test environments left on overnight"

- "Right-sizing > RI > Spot; fix waste before committing to reserved capacity"
- "Egress is the hidden cloud tax — architect to minimize cross-region and internet data transfer"
- "FinOps maturity: Inform (visibility) → Optimize (action) → Operate (culture); start with tagging"

---

39. Disaster Recovery & High Availability

Key Metrics

- **RTO (Recovery Time Objective)** → Max acceptable downtime after failure; minutes vs hours vs days; drives HA tier.
- **RPO (Recovery Point Objective)** → Max acceptable data loss; seconds vs hours; drives backup/replication strategy.
- **MTTR (Mean Time to Recovery)** → Average time to restore service; measure + improve.
- **MTBF (Mean Time Between Failures)** → Reliability measure; longer = more reliable.

DR Strategies (in order of cost/RTO)

- **Backup & Restore** → Cheapest; RPO = hours; RTO = hours; restore from backup to new environment.
- **Pilot Light** → Core infrastructure always on (DB); app scaled to zero; RPO = minutes; RTO = 10–30 min.
- **Warm Standby** → Scaled-down running copy; RPO = seconds/minutes; RTO = minutes; traffic switch.
- **Multi-Site Active/Active** → Full traffic in both regions simultaneously; RPO ≈ 0; RTO ≈ 0; most expensive.

High Availability Patterns

- **Multi-AZ** → Survive single AZ failure; synchronous replication; automatic failover; standard for production.
- **Multi-Region** → Survive regional failure; asynchronous or synchronous replication; complex; for critical systems.
- **Health Checks** → Route 53, ALB, NLB, Azure Front Door; route away from unhealthy endpoints.
- **Chaos Engineering** → Intentionally inject failures; validate HA; Chaos Monkey, AWS FIS, Azure Chaos Studio.

- **Runbooks** → Documented recovery procedures; tested regularly; reduce MTTR through preparation.
- **Game Days** → Scheduled DR exercises; team practices failover; validate RTO/RPO targets.

Interview Deep-Dive Keywords

- "RTO and RPO are business requirements — engineers implement the architecture that meets them"
- "Active/active is not just having two regions — data must be writable in both with conflict resolution"
- "Untested DR is no DR — if you haven't failed over recently, assume it won't work when needed"
- "Pilot light keeps DB replicated so data is current — app layer is the fast part to scale up"

---

40. Cloud Compliance & Governance

Compliance Frameworks

- **SOC 2** → Security, availability, processing integrity, confidentiality, privacy; Type I (design), Type II (operating effectiveness).
- **ISO 27001** → Information security management system; risk-based; certification.
- **PCI-DSS** → Payment card data; cardholder data environment (CDE); quarterly scans; encryption required.
- **HIPAA** → US healthcare PHI; Business Associate Agreement (BAA) with cloud provider required.
- **GDPR** → EU personal data; data processor agreement; data residency constraints.
- **FedRAMP** → US government cloud; authorized service providers; AWS GovCloud, Azure Government, GCP.

Cloud Governance Tools

- **AWS Control Tower + SCPs** → Landing zones; guardrails; prevent non-compliant resource creation.
- **Azure Policy + Blueprints** → Enforce config standards; auto-remediation; initiative assignments.
- **GCP Org Policy** → Constraint-based; restrict resource types, regions, service account keys.
- **Cloud Security Posture Management (CSPM)** → Defender for Cloud (Azure), Security Hub (AWS), SCC (GCP); automated compliance checks.

- **SIEM** → Sentinel (Azure), Security Lake + OpenSearch (AWS), Chronicle (GCP); aggregate security events.

Best Practices

- **Immutable Audit Logs** → CloudTrail (S3 + MFA delete), Azure Monitor export, GCP Log Router; tamper-proof.
- **Infrastructure Compliance as Code** → Checkov, tfsec, conftest; scan IaC before apply.
- **Least Privilege Enforcement** → Regular IAM reviews; IAM Access Analyzer; remove unused permissions.
- **Data Residency Controls** → Restrict resource creation to compliant regions via SCPs/policies.

Interview Deep-Dive Keywords

- "Compliance is a shared responsibility — cloud provider covers infrastructure compliance; customer covers data and config"
- "SOC 2 Type II report from your cloud provider covers the infrastructure layer — you still need to prove application controls"
- "Shift-left compliance: scan Terraform with Checkov before `terraform apply`; catch issues before they reach prod"
- "GDPR data residency: enforce via SCP/Org Policy to prevent resource creation in non-EU regions"

---

41. Cloud Anti-Patterns

Architecture Anti-Patterns

- **Lift-and-Shift Without Optimization** → Move VMs to cloud unchanged; pay cloud prices for on-prem patterns; no elasticity benefit.
- **Monolith in Containers** → Containerize monolith then claim microservices; still a big ball of mud; distributed monolith problems.
- **Snowflake Servers** → Manually configured unique servers; can't replace; blocks auto-scaling; use immutable AMIs/images.
- **Tightly Coupled Services** → Direct synchronous calls across services; cascade failures; use async messaging.
- **Missing Circuit Breaker** → No fallback when dependency fails; cascade failure brings down entire system.

- **Synchronous Everything** → Sync calls where async would do; tight temporal coupling; harder to scale.

Cost Anti-Patterns

- **Always-On Dev/Test** → Development environments running nights and weekends; schedule auto-shutdown.
- **Overprovisioned Instances** → 5% CPU utilization on large instance; never reviewed; right-size quarterly.
- **Manual Scaling** → Human adjusts capacity; always late; over-provision as safety buffer; use Auto Scaling.
- **No Tagging** → Can't allocate costs; can't find owners; can't automate cleanup.
- **Data Hoarding** → Store everything forever; no lifecycle policies; S3/storage costs compound.
- **Ignoring Egress** → Cross-region transfers between services; VPC endpoints not used; unnecessary internet egress.

Security Anti-Patterns

- **Public S3 Buckets / Blob Containers** → #1 cloud data breach vector; block public access at account level.
- **Hardcoded Credentials** → API keys in source code, Docker images, environment variables in plain text; use secrets manager.
- **Overpermissive IAM** → `*:*` policies; AdministratorAccess for service accounts; wildcard resource ARNs.
- **No MFA on Root/Admin** → Root account without MFA; single factor for console access.
- **Open Security Groups** → `0.0.0.0/0` inbound on all ports; common after debugging; left in production.
- **No Audit Logging** → CloudTrail/Activity Log disabled; no forensics capability after incident.

Operational Anti-Patterns

- **Manual Production Changes** → SSH into production; click-ops in console; no audit trail; config drift.
- **No Runbooks** → Incident response from memory; slow MTTR; inconsistent; document everything.
- **Alert Fatigue** → Too many low-value alerts; on-call ignores alerts; miss real incidents.
- **Untested DR** → DR plan exists on paper; never exercised; fails when needed; test quarterly.

Interview Deep-Dive Keywords

- "The most dangerous anti-pattern is the one you can't see — public S3 buckets and open SGs are invisible in large orgs without automation"
- "Config drift is how you end up with snowflake servers — IaC and immutable infrastructure prevent it"
- "Alert fatigue is an SRE emergency — prune ruthlessly; only page for things that need human intervention"
- "Lift-and-shift is fine for getting out of a data center; it's not fine as a final architecture"

---

42. Interview Master Keywords

Cloud Trade-offs

- **Managed vs Unmanaged** → Managed = less control, less ops overhead; Unmanaged = full control, full responsibility.
- **Serverless vs Containers** → Serverless = auto-scale to zero, cold starts, limits; Containers = consistent, portable, more ops.
- **Sync vs Async** → Sync = simple, tightly coupled, cascade failure risk; Async = resilient, decoupled, eventual consistency.
- **Multi-AZ vs Multi-Region** → Multi-AZ = HA (survive zone failure); Multi-Region = DR (survive regional failure); different costs.
- **Spot vs Reserved vs On-Demand** → Spot = cheapest, interruptible; Reserved = committed, discounted baseline; On-demand = flexible, expensive.
- **VPN vs Direct Connect** → VPN = fast to setup, variable latency, cheap; Direct Connect = consistent, dedicated, expensive.
- **Consistency vs Availability** → CAP theorem; strong consistency = CP; high availability = AP; design knowing the trade-off.

Decision Frameworks

- **Choose Cloud** → AWS = breadth + ecosystem; Azure = enterprise + Microsoft; GCP = data/ML + Kubernetes.
- **Choose Compute** → Lambda = event-driven, short-lived; Container = stateless long-running; VM = stateful, legacy.
- **Choose Storage** → Object (S3) = files/backups/data lake; Block (EBS) = OS/DB; File (EFS) = shared access.
- **Choose Database** → RDBMS = structured + ACID; DynamoDB = scale + simple access; Cosmos DB = global + multi-model; BigQuery = analytics.

- **Scale Strategy** → Read scale = replicas + cache; Write scale = sharding + partitioning; Stateless first.
- **HA Strategy** → Multi-AZ for 99.9%; Multi-Region for 99.99%; Active-Active for 99.999%.

One-Liners for Interviews

- "The shared responsibility model means the cloud is secure, but that doesn't mean your workload is"
- "Stateless is the prerequisite for elasticity — if you can't scale horizontally, you haven't finished the design"
- "Design for failure at every level — AZ, region, service dependency, network partition"
- "Right tool for the right job: Lambda for events, Fargate for containers, EC2 for VMs that need control"
- "Cost optimization starts with visibility — you can't optimize what you can't measure; tag everything"
- "IaC is the only acceptable way to manage cloud infrastructure at scale — manual changes are a liability"
- "Multi-cloud is a strategy, not a default — operational complexity must be justified by the business benefit"
- "CAP theorem in practice: partitions happen, so you choose between consistency and availability per service"
- "Egress fees are the hidden cloud bill — architect to keep data in the same region as the compute that needs it"
- "Auto scaling is not a magic fix — if your app isn't stateless and startup isn't fast, scaling won't save you"